



Material Design Layout

& Jetpack Compose

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet
2025/2026

Cos'è il Material Design

Material Design è il sistema di progettazione di Google che fornisce linee guida, componenti e strumenti per costruire interfacce coerenti, accessibili e riconoscibili su Android, iOS, web e Flutter.

1

Material is the metaphor

Le superfici e l'inchiostro digitali si comportano come oggetti fisici reali con luce, ombra e movimento.

2

Bold, graphic, intentional

Tipografia marcata, griglie, immagini grandi e colori intenzionali guidano l'utente.

3

Motion provides meaning

Le transizioni e le animazioni non sono ornamenti: comunicano relazioni e gerarchia.



Cosa ti dà Material

- ✓ Linee guida ufficiali e pubbliche
- ✓ Libreria di componenti pronti
- ✓ Theming centralizzato (colori, type)
- ✓ Coerenza con altre app Android
- ✓ Best practice di accessibilità

Setup del progetto

Per usare Material Design occorre la dipendenza nel modulo app e un tema Material come parent.

build.gradle.kts (modulo :app)

```
dependencies {  
    // Material Components (XML)  
    implementation(  
        "com.google.android.material:material:1.12.0")  
    // Compose Material 3  
    implementation(  
        "androidx.compose.material3:material3:1.3.0")  
}
```

Kotlin

res/values/themes.xml

```
<style name="Theme.MyApp"  
    parent="Theme.Material3.DayNight">  
    <item name="colorPrimary">  
        @color/indigo_700</item>  
    <item name="colorSecondary">  
        @color/amber_500</item>  
</style>
```

XML

Tip

Il parent del tema deve essere Theme.Material3.* (o Theme.MaterialComponents.* per progetti legacy).

Il sistema di colori Material 3

Material 3 organizza i colori in ruoli semantici. Definendoli una volta, tutta la UI si adatta.

primary #0F3D91	secondary #1E5BD6	tertiary #E65100	surface #F7F8FA	error #B00020	outline #DADCE5
-------------------------------	---------------------------------	--------------------------------	-------------------------------	-----------------------------	-------------------------------

Accesso nel codice Compose

```
Text(  
    text = "Titolo",  
    color = MaterialTheme.colorScheme.primary,  
    style = MaterialTheme.typography.titleLarge  
)
```

Kotlin

Tipografia e shape

Material 3 definisce una scala tipografica e un sistema di shape. Si usano per riferimento semantico.

Scala tipografica

<code>displayLarge</code>	Aa Material
<code>headlineMedium</code>	Aa Material
<code>titleLarge</code>	Aa Material
<code>bodyLarge</code>	Aa Material
<code>labelSmall</code>	Aa Material

Shape tokens



Due approcci per costruire l'interfaccia

Su Android si hanno due modi di descrivere i layout: il sistema classico basato su View/XML e Jetpack Compose, l'approccio dichiarativo in Kotlin.



Sistema View / XML

L'approccio storico, ancora largamente usato

- Layout descritti in file .xml
- ViewBinding o findViewById
- Material Components for Android
- Adatto a progetti esistenti



Jetpack Compose

Toolkit moderno, dichiarativo, in Kotlin puro

- UI definita con funzioni @Composable
- Stato osservabile e ricomposizione automatica
- Material 3 (androidx.compose.material3)
- Raccomandato per nuovi progetti

Il sistema di layout di Android

Un layout è un albero di View. Ogni ViewGroup decide come disporre i propri figli. Conoscere i ViewGroup principali è il primo passo.



ConstraintLayout

Posiziona le View tramite vincoli relativi tra loro o al parent. Il più flessibile e performante.



LinearLayout

Dispone i figli in una riga (horizontal) o colonna (vertical). Semplice ma limitato.



FrameLayout

Sovrappone i figli sullo stesso punto. Ideale per stack di elementi e per fragment container.



RecyclerView

Lista virtualizzata efficiente per molti elementi omogenei (adapter + ViewHolder).

ConstraintLayout in pratica

Ogni View deve avere almeno un vincolo orizzontale e uno verticale. I vincoli si esprimono con `app:layout_constraint*_to*Of`.

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/title"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Benvenuto"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <Button
        android:id="@+id/cta"
        android:text="Inizia"
        app:layout_constraintTop_toBottomOf="@id/title"
        app:layout_constraintEnd_toEndOf="parent"/>
</androidx.constraintlayout.widget.ConstraintLayout>
```

Kotlin / XML

Anteprima schematica

Collegare il layout a Kotlin: ViewBinding

ViewBinding genera una classe per ogni layout XML; nessun cast.

1. Abilita ViewBinding nel build.gradle.kts

```
android {  
    buildFeatures { viewBinding = true }  
}
```

Kotlin / XML

Perché ViewBinding?

- Type-safe: niente cast a runtime
- Null-safe: viste opzionali sono nullable
- Niente findViewById sparsi
- Generato per ogni file di layout

2. Usa la classe generata MainActivityBinding

```
class MainActivity : AppCompatActivity() {  
  
    private lateinit var binding:  
        ActivityMainBinding  
  
    override fun onCreate(state: Bundle?) {  
        super.onCreate(state)  
        binding = ActivityMainBinding  
            .inflate(layoutInflater)  
            setContentView(binding.root)  
  
        binding.title.text = "Benvenuto"  
        binding.cta.setOnClickListener {  
            // gestisci il click  
        }  
    }  
}
```

Kotlin / XML

ConstraintLayout in XML

Ogni View deve avere almeno un vincolo orizzontale e uno verticale. È il layout più flessibile e performante.

```
<ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/title"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Benvenuto"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <Button
        android:id="@+id/cta"
        android:text="Inizia"
        app:layout_constraintTop_toBottomOf="@id/title"
        app:layout_constraintEnd_toEndOf="parent"/>
</ConstraintLayout>
```

XML

*Anteprima***Benvenuto****Inizia**

AppBar e BottomNavigation (XML)

I componenti Material per XML si trovano nel package `com.google.android.material`.

MaterialToolbar in XML

```
<com.google.android.material.appbar.MaterialToolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="?attr/actionBarSize"
    app:title="La mia app"
    app:navigationIcon="@drawable/ic_menu"
    app:menu="@menu/top_menu"
    style="@style/Widget.Material3.Toolbar"/>
```

XML

BottomNavigationView in XML

```
<com.google.android.material.bottomnavigation
    .BottomNavigationView
    android:id="@+id/bottomNav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:menu="@menu/bottom_nav"
    app:labelVisibilityMode="labeled"
    android:layout_gravity="bottom"/>
```

XML

```
binding.bottomNav.setOnItemClickListener { item ->
    when (item.itemId) {
        R.id.nav_home -> { showFragment(HomeFragment()); true }
        R.id.nav_search -> { showFragment(SearchFragment()); true }
        else -> false
    }
}
```

Kotlin

MaterialCardView e TextInputLayout

Card e TextField sono i componenti più usati per raggruppare contenuti e raccogliere input.

MaterialCardView

```
<com.google.android.material.card
    .MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardElevation="2dp"
    app:cardCornerRadius="12dp">
    <LinearLayout
        android:orientation="vertical"
        android:padding="16dp" ...>
        <TextView android:text="Titolo"
            android:textAppearance=
                "?attr/textAppearanceTitleLarge"/>
        <TextView android:text="Corpo..."
            android:textAppearance=
                "?attr/textAppearanceBodyMedium"/>
    </LinearLayout>
</com.google.android.material.card
    .MaterialCardView>
```

XML

TextInputLayout

```
<com.google.android.material.textfield
    .TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Email"
    app:helperText="Inserisci un indirizzo valido"
    app:endIconMode="clear_text"
    style="@style/Widget.Material3
        .TextInputLayout.OutlinedBox">
    <com.google.android.material.textfield
        .TextInputEditText
        android:id="@+id/email"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress"/>
</com.google.android.material.textfield
    .TextInputLayout>
```

XML

Jetpack Compose: panoramica

Compose è il toolkit dichiarativo di Android per costruire UI in Kotlin puro. È il futuro raccomandato da Google.

100% Kotlin

Niente XML, niente findViewById. La UI è funzione dello stato.



Reattivo

Quando lo stato cambia, solo i composabile interessati si ricalcolano.



“Composabile”

Piccole funzioni @Composable si combinano come mattoncini.



Material 3 integrato

androidx.compose.material3 fornisce tutti i componenti.

Il primo schermo in Compose

In Compose la UI è una funzione Kotlin annotata con `@Composable`. Si invoca in `setContent { }`.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(b: Bundle?) {  
        super.onCreate(b)  
        setContent {  
            MyAppTheme {  
                Surface(  
                    color = MaterialTheme  
                        .colorScheme.background  
                ) {  
                    Greeting("Studente")  
                }  
            }  
        }  
    }  
}  
  
@Composable  
fun Greeting(name: String) {  
    Text(  
        text = "Ciao, $name!",  
        style = MaterialTheme.typography.headlineMedium  
    )  
}
```

Kotlin

Tre passaggi chiave

1. Il tema

MyAppTheme avvolge l'app: colori, typography, shape.

2. Surface

Superficie con colore di sfondo dal tema.

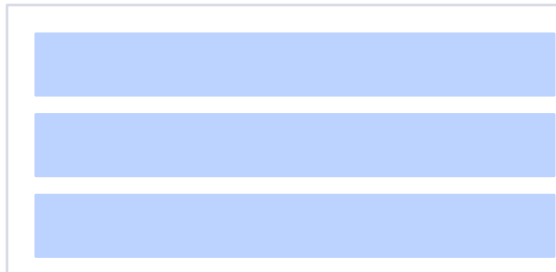
3. @Composable

Funzioni che descrivono la UI. Ricevono stato come parametri.

Column, Row, Box

Tre composabile base per la quasi totalità dei layout: verticale, orizzontale, sovrapposto.

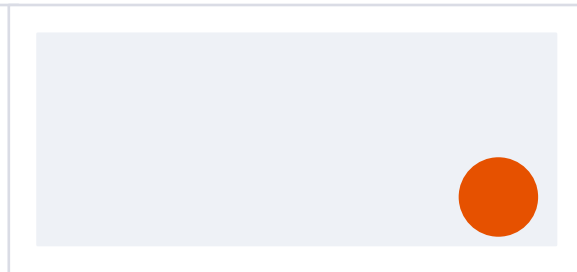
Column



Row



Box



```
Kotlin
Column(
    verticalArrangement =
        Arrangement.spacedBy(8.dp)
) {
    Text("Item 1")
    Text("Item 2")
    Text("Item 3")
}
```

Kotlin

```
Kotlin
Row(
    horizontalArrangement =
        Arrangement.SpaceBetween
) {
    Text("A")
    Text("B")
    Text("C")
}
```

Kotlin

```
Kotlin
Box(
    contentAlignment =
        Alignment.BottomEnd
) {
    Image(...)
    FloatingActionButton(
        onClick = { }
    ) { Icon(...) }
}
```

Kotlin

Modifier: styling e layout

Modifier è una catena di trasformazioni che descrive padding, dimensioni, click, sfondo, bordi e altro.

```
@Composable
fun ProfileCard(name: String) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
            .clickable { /* navigare al profilo */ },
        elevation = CardDefaults.cardElevation(4.dp)
    ) {
        Row(
            modifier = Modifier.padding(12.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Icon(
                Icons.Default.Person,
                contentDescription = "Avatar",
                modifier = Modifier.size(48.dp)
            )
            Spacer(Modifier.width(12.dp))
            Text(
                name,
                style = MaterialTheme.typography.titleMedium
            )
        }
    }
}
```

Kotlin

Regole dei Modifier

- Sempre primo parametro nominato
- L'ordine conta: padding poi background ≠ background poi padding
- fillMaxWidth() per larghezza piena
- size() per dimensione fissa
- clickable {} per rendere interattivo
- clip() per arrotondare i bordi

Stato e ricomposizione

In Compose la UI è funzione dello stato. Quando lo stato cambia, i composabile che lo leggono si ricalcolano.

<https://developer.android.com/develop/ui/compose/state>

```
@Composable
fun Counter() {
    var count by remember {
        mutableStateOf(0)
    }
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        horizontalAlignment =
            Alignment.CenterHorizontally
    ) {
        Text(
            "Hai cliccato $count volte",
            style = MaterialTheme.typography.titleLarge
        )
        Spacer(Modifier.height(12.dp))
        Button(onClick = { count++ }) {
            Text("Incrementa")
        }
    }
}
```

Kotlin

Come funziona

remember

Mantiene il valore tra le ricomposizioni.

mutableStateOf

Crea un'istanza osservabile. La modifica innesca la ricomposizione.

by (delegated property)

Rende la sintassi più pulita: count++ invece di count.value++.

State hoisting

Il pattern state hoisting sposta lo stato verso l'alto: il composable diventa stateless e riutilizzabile.

```
// Stateless: riceve stato e callback
@Composable
fun CounterDisplay(
    count: Int,
    onIncrement: () -> Unit
) {
    Column(horizontalAlignment =
        Alignment.CenterHorizontally
    ) {
        Text("Contatore: $count")
        Button(onClick = onIncrement) {
            Text("Incrementa")
        }
    }
}

// Stateful: possiede lo stato
@Composable
fun CounterScreen() {
    var count by remember { mutableStateOf(0) }
    CounterDisplay(count) { count++ }
}
```

Kotlin

Perché fare hoisting?

- Riutilizzo: lo stesso composable con dati diversi
- Test: puoi passare valori fissi nei test
- Preview: funziona in @Preview con dati mock
- Single source of truth: lo stato vive in un solo posto
- ViewModel-ready: lo stato può vivere nel VM

TextField e OutlinedTextField

I campi di testo in Compose sono controllati: il valore è nello stato, non nel widget.

```
@Composable
fun LoginForm() {
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }

    Column(Modifier.padding(16.dp)) {
        OutlinedTextField(
            value = email,
            onChange = { email = it },
            label = { Text("Email") },
            leadingIcon = {
                Icon(Icons.Default.Email, null)
            },
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(Modifier.height(8.dp))
        OutlinedTextField(
            value = password,
            onChange = { password = it },
            label = { Text("Password") },
            visualTransformation =
                PasswordVisualTransformation(),
            modifier = Modifier.fillMaxWidth()
        )
    }
}
```

Kotlin

Differenze dal sistema View

- value + onChange: pattern controllato
- label è un composable, non una stringa
- leadingIcon / trailingIcon per icone
- visualTransformation per nascondere testo
- isError per mostrare lo stato di errore

BottomNavigationView (sistema View)

Permette di navigare fra 3-5 destinazioni di pari livello. In XML si dichiara il widget e un menu di voci.

res/menu/bottom_nav.xml

```
<menu xmlns:android=
    "http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/nav_home"
        android:icon="@drawable/ic_home"
        android:title="Home"/>
    <item
        android:id="@+id/nav_search"
        android:icon="@drawable/ic_search"
        android:title="Cerca"/>
    <item
        android:id="@+id/nav_profile"
        android:icon="@drawable/ic_person"
        android:title="Profilo"/>
</menu>
```

Kotlin / XML

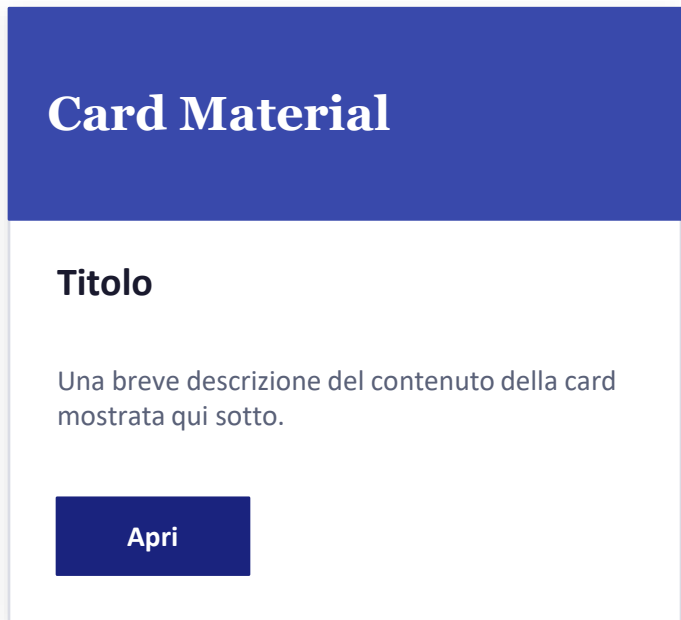
MainActivity.kt

```
binding.bottomNav
    .setOnItemSelectedListener { item ->
        when (item.itemId) {
            R.id.nav_home -> {
                mostraFragment(HomeFragment())
                true
            }
            R.id.nav_search -> {
                mostraFragment(SearchFragment())
                true
            }
            R.id.nav_profile -> {
                mostraFragment(ProfileFragment())
                true
            }
            else -> false
        }
    }
```

Kotlin / XML

MaterialCardView

Le Card sono superfici elevate che raggruppano informazioni correlate e azioni. Hanno elevation, shape e ripple coerenti con il tema.



```
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardElevation="2dp"
    app:cardCornerRadius="12dp">

    <LinearLayout
        android:orientation="vertical"
        android:padding="16dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <TextView
            android:text="Titolo"
            android:textAppearance=
                "?attr/textAppearanceTitleLarge"/>

        <TextView
            android:text="Descrizione..."
            android:textAppearance=
                "?attr/textAppearanceBodyMedium"/>

    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
```

Kotlin / XML

Input testuali: TextInputLayout

TextInputLayout aggiunge label fluttuante, helper text ed error state al classico EditText.

```
Kotlin / XML
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Email"
    app:helperText="Inserisci un indirizzo valido"
    app:endIconMode="clear_text"
    style=
        "@style/Widget.Material3.TextInputLayout.OutlinedBox">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/email"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress"/>
</com.google.android.material.textfield.TextInputLayout>
```

```
Kotlin / XML
// Validazione
fun valida(): Boolean {
    val v = binding.email.text
        .toString().trim()
    return if (v.isEmpty()) {
        binding.emailLayout.error =
            "Campo obbligatorio"
        false
    } else if (!Patterns.EMAIL_ADDRESS
        .matcher(v).matches()) {
        binding.emailLayout.error =
            "Email non valida"
        false
    } else {
        binding.emailLayout.error = null
        true
    }
}
```

Scaffold: struttura della schermata

Scaffold organizza TopAppBar, BottomBar, FAB, Drawer e contenuto in un layout coerente con Material 3.

```
@Composable
fun HomeScreen() {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("La mia app") },
                navigationIcon = {
                    IconButton(onClick = { }) {
                        Icon(Icons.Default.Menu, "Menu")
                    }
                },
                actions = {
                    IconButton(onClick = { }) {
                        Icon(Icons.Default.Search, "Cerca")
                    }
                }
            )
        },
        floatingActionButton = {
            FloatingActionButton(onClick = { }) {
                Icon(Icons.Default.Add, "Aggiungi")
            }
        },
        bottomBar = {
            NavigationBar { /* NavigationBarItem ... */ }
        }
    ) { padding ->
        ContentList(Modifier.padding(padding))
    }
}
```

Kotlin

Card e ElevatedCard in Compose

Le Card raggruppano contenuti correlati con elevazione, bordi arrotondati e ripple effect coerenti col tema.

```
@Composable
fun ArticleCard(article: Article) {
    ElevatedCard(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        elevation = CardDefaults
            .cardElevation(defaultElevation = 4.dp)
    ) {
        Column(Modifier.padding(16.dp)) {
            Text(
                article.title,
                style = MaterialTheme.typography.titleLarge
            )
            Spacer(Modifier.height(8.dp))
            Text(
                article.summary,
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                maxLines = 3,
                overflow = TextOverflow.Ellipsis
            )
            Spacer(Modifier.height(12.dp))
            Row {
                AssistChip(
                    onClick = { },
                    label = { Text(article.category) }
                )
            }
        }
    }
}
```

Kotlin

Titolo articolo

Sommario del contenuto della card con testo body medium che può andare su più righe...

Tech

LazyColumn: liste efficienti

LazyColumn è l'equivalente di RecyclerView: compone solo gli elementi visibili sullo schermo.

```
data class Student(  
    val id: Int,  
    val name: String,  
    val matricola: String  
)  
  
@Composable  
fun StudentList(students: List<Student>) {  
    LazyColumn(  
        contentPadding = PaddingValues(12.dp),  
        verticalArrangement =  
            Arrangement.spacedBy(8.dp)  
    ) {  
        items(  
            items = students,  
            key = { it.id }  
        ) { student ->  
            StudentRow(student)  
        }  
    }  
}
```

Kotlin

```
@Composable  
fun StudentRow(s: Student) {  
    Card(  
        Modifier.fillMaxWidth()  
    ) {  
        Column(  
            Modifier.padding(12.dp)  
        ) {  
            Text(  
                s.name,  
                style = MaterialTheme  
                    .typography.titleMedium  
            )  
            Text(  
                s.matricola,  
                color = MaterialTheme  
                    .colorScheme.outline  
            )  
        }  
    }  
}
```

Kotlin

LazyVerticalGrid

Per layout a griglia: LazyVerticalGrid con colonne fisse o adattive.

```
@Composable
fun PhotoGrid(photos: List<Photo>) {
    LazyVerticalGrid(
        columns = GridCells.Adaptive(minSize = 128.dp),
        contentPadding = PaddingValues(8.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp),
        horizontalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        items(photos, key = { it.id }) { photo ->
            AsyncImage(
                model = photo.url,
                contentDescription = photo.title,
                modifier = Modifier.aspectRatio(1f).clip(RoundedCornerShape(8.dp)),
                contentScale = ContentScale.Crop
            )
        }
    }
}
```

Kotlin



GridCells.Fixed vs GridCells.Adaptive

Fixed(3) = sempre 3 colonne. Adaptive(128.dp) = quante colonne ci stanno con almeno 128 dp di larghezza.

Navigazione con NavHost

Jetpack Navigation Compose gestisce le schermate come routes con un NavController.

```
@Composable
fun AppNavigation() {
    val navController = rememberNavController()
    NavHost(
        navController = navController,
        startDestination = "home"
    ) {
        composable("home") {
            HomeScreen(
                onNavigateToDetail = { id ->
                    navController.navigate("detail/$id")
                }
            )
        }
        composable(
            "detail/{itemId}",
            arguments = listOf(navArgument("itemId") { type = NavType.IntType })
        ) { backStackEntry ->
            val itemId = backStackEntry.arguments?.getInt("itemId") ?: 0
            DetailScreen(itemId)
        }
    }
}
```

Kotlin

NavigationBar in Compose

NavigationBar + NavigationBarItem sostituiscono BottomNavigationView del sistema View.

```
sealed class Screen(val route: String, val label: String, val icon: ImageVector) {  
    object Home : Screen("home", "Home", Icons.Default.Home)  
    object Search : Screen("search", "Cerca", Icons.Default.Search)  
    object Profile : Screen("profile", "Profilo", Icons.Default.Person)  
}
```

Kotlin

```
@Composable  
fun BottomNavBar(navController: NavController) {  
    val items = listOf(Screen.Home, Screen.Search, Screen.Profile)  
    val currentRoute = navController  
        .currentBackStackEntryAsState().value?.destination?.route  
  
    NavigationBar {  
        items.forEach { screen ->  
            NavigationBarItem(  
                icon = { Icon(screen.icon, screen.label) },  
                label = { Text(screen.label) },  
                selected = currentRoute == screen.route,  
                onClick = {  
                    navController.navigate(screen.route) {  
                        popUpTo(navController.graph.startDestinationId) { saveState = true }  
                        launchSingleTop = true; restoreState = true  
                    }  
                }  
            )  
        }  
    }  
}
```

Adaptive: telefoni, tablet, foldable

Material 3 definisce window size classes. Adattiamo il layout in base allo spazio disponibile.



Compact

< 600 dp

Telefoni portrait. Colonna singola, BottomNavigation.



Medium

600 – 840 dp

Tablet piccoli / foldable aperti. NavigationRail laterale.



Expanded

> 840 dp

Tablet grandi / desktop. Pannello list-detail, Drawer permanente.

```
val windowSize = calculateWindowSizeClass(activity)
when (windowSize.widthSizeClass) {
    WindowWidthSizeClass.Compact -> CompactLayout()
    WindowWidthSizeClass.Medium  -> MediumLayout()
    WindowWidthSizeClass.Expanded -> ExpandedLayout()
}
```

Kotlin

Accessibilità: requisiti minimi

Material Design ha l'accessibilità nel proprio DNA. Sono requisiti, non extra opzionali.



Tocco minimo 48 × 48 dp

Tutti gli elementi interattivi devono garantire un'area di tocco di almeno 48 dp.



Contrasto colore $\geq 4.5:1$

Il testo normale deve avere contrasto sufficiente con lo sfondo.



contentDescription per icone

Ogni Icon/Image deve avere contentDescription (null solo se decorativa).



Supporto testo grande

Usare sp per il testo, non dp. Testare con impostazioni di sistema al massimo.

```
Icon(Icons.Default.Add, contentDescription = "Aggiungi nuovo elemento") // OK
```

Kotlin



contentDescription = null solo per icone puramente decorative che non aggiungono informazione.

Tema scuro (Dark Theme)

Material 3 supporta light e dark theme nativamente. In Compose si gestisce con `isSystemInDarkTheme()`.

```
@Composable
fun MyAppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    val colorScheme = if (darkTheme) {
        darkColorScheme(
            primary = Color(0xFF90CAF9),
            secondary = Color(0xFFFFCC80),
            background = Color(0xFF121212)
        )
    } else {
        lightColorScheme(
            primary = Color(0xFF0F3D91),
            secondary = Color(0xFFE65100),
            background = Color(0xFFFFFFFF)
        )
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = AppTypography,
        content = content
    )
}
```

Kotlin

XML vs Compose: confronto

Entrambi gli approcci sono validi. Ecco un confronto pratico per orientarsi.

Aspetto	Sistema View / XML	Jetpack Compose
Linguaggio UI	XML + Kotlin	Kotlin puro
Paradigma	Imperativo	Dichiarativo
Stato	manuale (setText...)	reattivo (mutableStateOf)
Preview	Layout Editor	@Preview in codice
Temi	themes.xml	MaterialTheme { }
Liste	RecyclerView + Adapter	LazyColumn / LazyGrid
Navigazione	Fragment + NavGraph XML	NavHost composabile
Curva di apprendimento	familiare per Java devs	richiede mentalità funzionale
Raccomandazione Google	progetti legacy	nuovi progetti

Best practice vs anti-pattern

DA FARE

- Usa `MaterialTheme.colorScheme` e `typography`
- Preferisci `Compose` per nuovi progetti
- Estrai composabile piccoli e riutilizzabili
- State hoisting: stato verso l'alto
- `contentDescription` per tutte le icone
- Testa con dark theme e testo grande
- Usa `LazyColumn` con key per le liste

DA EVITARE

- Colori hard-coded sparsi nei layout
- `LinearLayout` annidati a profondità 4+
- Stato dentro il composabile senza hoisting
- Margini fissi in pixel (px) anziché dp/sp
- Icone senza `contentDescription`
- `GlobalScope` per operazioni UI-related
- Ricreare liste intere senza key

Errori comuni e soluzioni

remember dimenticato

Lo stato si resetta a ogni ricomposizione. Avvolgi in `remember { }`.

LazyColumn senza key

Senza key il diffing è inefficiente e causa glitch nelle animazioni.

Composable troppo grandi

Un composable da 200 righe è un red flag. Estrai sotto-composable.

Modifier.padding dopo .clickable

Il padding interno non è cliccabile. Metti `clickable` dopo il padding se vuoi l'area estesa.

mutableListOf non osservabile

Usa `mutableStateListOf()` o `toMutableStateList()` per liste reattive.

Effetti nel corpo del composable

Operazioni con side effect vanno in `LaunchedEffect` o `SideEffect`.

Cheat-sheet Compose

I pattern più frequenti in un colpo d'occhio.

```
// 1. Stato
var x by remember { mutableStateOf(0) }

// 2. Lista
LazyColumn { items(list, key = { it.id }) { item -> ItemRow(item) } }

// 3. Navigazione
navController.navigate("detail/${id}")

// 4. Tema
MaterialTheme.colorScheme.primary
MaterialTheme.typography.titleLarge

// 5. Scaffold
Scaffold(topBar = { ... }, floatingActionButton = { ... }) { padding -> ... }

// 6. Modifier essenziali
Modifier.fillMaxWidth().padding(16.dp).clickable { }

// 7. Effetti
LaunchedEffect(key) { /* coroutine */ }
DisposableEffect(key) { onDispose { /* cleanup */ } }

// 8. Input
OutlinedTextField(value = text, onValueChange = { text = it })
```

Kotlin